

Relatório Técnico de projeto - Módulo Android

VitalRoute

Disciplina:	Universidade de Aveiro - Computação Móvel
Data:	18 de maio de 2026
Alunos:	114421: Duarte Lourenço 120172: João Silva
Resumo do projeto:	VitalRoute é uma aplicação de fitness e segurança pessoal que permite gravar atividades ao ar livre (corrida, caminhada e ciclismo). Tem monitorização GPS em tempo real, deteção automática de quedas e envio de alertas SMS para contactos de emergência.

Conteúdos do

relatório:

[1 Conceito da](#)

[aplicação](#)

[Visão Geral](#)

[Mapa de jornada essencial](#)

[2 Solução implementada](#)

[Desing técnico](#)

[Visão geral da arquitetura de software](#)

[Estrutura da aplicação Android](#)

[Estratégia de dados](#)

[Estratégias avançadas de design](#)

[Interações implementadas](#)

[Limitações do projeto](#)

[3 Conclusões e recursos de suporte](#)

[Lições aprendidas](#)

[Distribuição do trabalho dentro da equipa](#)

[Recursos do projeto \(links diretos\)](#)

1 Conceito da Aplicação

Visão Geral

O VitalRoute é uma aplicação Android direcionada para utilizadores que praticam atividades ao ar livre como corrida, caminhada ou ciclismo. A aplicação diferencia-se pela segurança: monitoriza a atividade, deteta quedas ou imobilidade prolongada e emite alertas via SMS para os contactos de emergência pré-definidos.

Os principais benefícios são a gravação automática e persistente de atividades, alertas SOS com localização e visualização do percurso com identificação de ciclovias.

Mapa de jornada essencial

Cenário: Ciclista que sai para um treino sozinho.

Fase	Ação do utilizador	Touchpoint na app	Resultado
Preparação	Abre a app, verifica o dashboard	HomeScreen	Vê estatísticas semanais, objetivos, GPS e meteorologia
Planeamento	Consulta o mapa da área	MapsScreen	Visualiza camada CyclOSM, verifica temperatura/vento via Open-Meteo
Início	Inicia gravação de atividade	RecordingScreen	Foreground service ativo: GPS, acelerómetro e geofencing iniciam
Durante	Pedala – app em segundo plano	RecordingService	Grava pontos GPS, calcula distância/velocidade/elevação, monitoriza quedas
Incidente	Cai da bicicleta	RecordingService	Deteção de queda -> countdown SOS -> SMS com localização enviado aos contactos
Chegada a zona segura	Chega a casa	RecordingService(geofencing)	Notificação automática aos contactos de chegada em segurança

Fim	Para a gravação	RecordingScreen	Atividade guardada no Firestore
Análise	Revê o treino	DiaryScreen/ActivityDetail	Visualiza rota, gráfico de elevação, estatísticas detalhadas

2 Solução implementada

Design técnico

Visão geral da arquitetura de software

A aplicação segue uma arquitetura em camadas com separação entre UI, lógica de negócio e dados:

UI Layers (Jetpack Compose) Screens + ViewModels (StateFlow)
Domain / Data Layer FirestoreRepository WeatherRepository NominatimRepository SosManager
External Services / Hardware Firebase Auth Firestore Open-Meteo OSMDroid GPS Acelerómetro BLE/GATT Nominatim SMS

Módulos principais:

RecordingService	Foreground service: GPS, acelerómetro, geofencing, SOS
FirestoreRepository	Toda a persistência e leitura de dados no Firestore

SosManager	Envio de SMS de emergência com localização GPS
BleGattManager	Conexão e leitura de dados de wearables BLE/GATT
WeatherRepository	Consulta de meteorologia via API Open-Meteo
NominatimRepository	Geocodificação de endereços
ActivityExporter	Exportação de atividades em formato CSV e GPX

Serviços externos e bibliotecas:

Serviço/Biblioteca	Versão	Propósito
Firebase Auth	BOM 33.1.0	Autenticação com verificação de email
Firebase Firestore	BOM 33.1.0	Base de dados em tempo real
OSMDroid	6.1.18	Renderização de mapas OpenStreetMap
Open-Meteo API	—	Dados meteorológicos (livre, sem chave)
Nominatim (OSM)	—	Geocodificação de endereços
Retrofit 2	2.9.0	Cliente HTTP para APIs REST
Play Services Location	21.0.1	FusedLocationProvider (GPS)

Sensores e hardware utilizados:

- GPS — LocationManager + FusedLocationProvider para rastreamento de rota
- Acelerómetro — SensorManager / SensorEventListener para deteção de quedas (algoritmo de 2 fases) e imobilidade
- Bluetooth BLE (GATT) — BleGattManager para conexão a wearables com suporte a Heart

Rate (0x180D), Free Fall (0x2A79), Acceleration 3D (0x2713) e perfil custom VitalRoute (0xFF00)

Estrutura da aplicação Android

O projeto segue o padrão MVVM com Repository Pattern:

- View — Ecrãs implementados com JetPack Compose. Cada ecrã é uma função *@Composable* que observa o estado exposto pelo ViewModel
- ViewModel — 9 ViewModels (um por módulo funcional). Expõem o estado através de *StateFlow<UiState>*, garantindo reatividade.
- Repository — 3 repositórios (*FirestoreRepository*, *WeatherRepository*, *NominatimRepository*) que abstraem a origem dos dados.

Estratégia de dados

Modelo de dados no Firestore:

Coleção	Campos principais
activities	id, type, startTime, endTime, distanceKm, durationSeconds, avgSpeedKmh, maxSpeedKmh, elevationM, calories, routePoints[], elevationPoints[]
contacts	id, name, relation, phone, sosEnabled, zonesEnabled
safeZones	id, name, address, lat, lng, radiusM, color
settings/main	fallSensitivity, sosCountdownSecs, immobilityAlertEnabled, immobilityMinutes, arrivalAlertEnabled, routeDeviationEnabled, weeklyGoalKm, metricSystem
liveLocation/current	lat, lng, speedKmh, distKm, updatedAt, isSharing

(doc raiz do utilizador)	uid, name, email, weightKg, heightCm, gender
--------------------------	--

Estratégias avançadas de design

1. Detecção de quedas com algoritmo de 2 fases

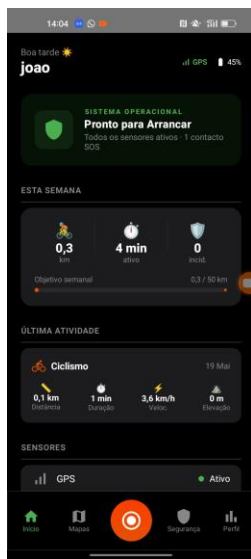
O acelerómetro é lido continuamente via `SensorManager / SensorEventListener` durante a gravação. O algoritmo opera em estados sequenciais: `NONE` -> `FREE_FALL` -> `IMPACT`. A transição para `FREE_FALL` ocorre quando a magnitude do vetor de aceleração desce abaixo de $\sim 4 \text{ m/s}^2$; a transição para `IMPACT` exige um pico acima de 30 m/s^2 nos segundos seguintes. Só a sequência completa aciona o SOS — um impacto isolado ou uma queda livre sem impacto não disparam alerta. A sensibilidade é configurável pelo utilizador (0..1), ajustando os thresholds dinamicamente. Após deteção, um countdown configurável (padrão 15s) permite cancelar; se não for cancelado, o `SosManager` envia SMS com coordenadas GPS reais via `SmsManager`.

2. Geofencing manual sem Google Geofencing API

A Google Geofencing API impõe um limite de 100 geofences por aplicação e introduz dependência de Play Services, indisponível em alguns dispositivos Android. O geofencing foi implementado de raiz no `RecordingService`: a cada atualização GPS, calcula-se a distância entre a posição atual e o centro de cada zona segura guardada no `Firestore`. Quando a distância desce abaixo do raio configurado (`radiusM`), a chegada é detetada e é enviada notificação aos contactos com `zonesEnabled = true`. Esta abordagem elimina dependências externas e dá controlo total sobre frequência de verificação e lógica de entrada/saída de zona.

Interações implementadas

Gravação de atividade com segurança ativa



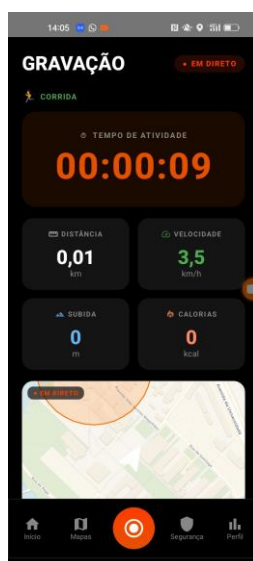
O dashboard apresenta o estado do sistema de segurança, estatísticas semanais e a última atividade. O botão central da barra de navegação acede ao ecrã de gravação.



O utilizador seleciona o tipo de atividade (Ciclismo, Corrida ou Caminhada) e pode definir um destino, que gera uma rota via OSRM visível durante a gravação.



O toggle de partilha de localização envia a posição em tempo real para os contactos SOS via Firestore. O slider "Deslizar para SOS" está sempre acessível.



Com a gravação ativa, o RecordingService corre como Foreground Service mantendo o GPS e acelerómetro ativos mesmo com o ecrã desligado. As métricas e o mapa atualizam em tempo real.



Após deslizar o slider (ou deteção automática de queda), a app entra em modo de emergência com contador regressivo de 15s (pode ser alterado nas definições). Se não cancelado, o SosManager envia SMS com localização GPS a todos os contactos de emergência.

Limitações do projeto

Integração BLE com deteção de quedas

O BleGattManager estabelece ligação GATT e lê dados de aceleração de wearables com suporte aos perfis Heart Rate (0x180D), Free Fall (0x2A79), Acceleration 3D (0x2713) e ao perfil custom VitalRoute (0xFF00). No entanto, a fusão entre os dados do wearable e o algoritmo de 2 fases do acelerómetro interno não está completa — em dispositivos sem perfil dedicado, a deteção de quedas recai exclusivamente no sensor do telemóvel.

Sem modo offline completo

O Firestore tem cache offline nativa que permite leituras e escritas sem conectividade, sincronizando automaticamente quando a ligação é restaurada. No entanto, o registo de conta e o primeiro login dependem obrigatoriamente do Firebase Auth, que requer ligação à internet.

3 Conclusões e recursos de suporte

Lições aprendidas

1. Foreground Service com múltiplos sensores em simultâneo

Manter GPS, acelerómetro e escrita no Firestore no mesmo serviço exigiu separação explícita de corrotinas por dispatcher: processamento de sensores em `Dispatchers.Default`, operações de rede em `Dispatchers.IO`. Sem esta separação, picos de escrita no Firestore atrasavam a amostragem do acelerómetro e introduziam latência na deteção de quedas. A frequência de leitura dos sensores também teve de ser calibrada para equilibrar precisão e consumo de bateria em atividades longas.

2. Algoritmo de deteção de quedas: reduzir falsos positivos sem perder sensibilidade

Uma deteção por threshold simples disparava SOS ao pousar o telemóvel numa mesa ou ao tropeçar. A solução foi o algoritmo de 2 fases — queda livre (magnitude $< 4 \text{ m/s}^2$) seguida obrigatoriamente de impacto (magnitude $> 30 \text{ m/s}^2$) — garantindo que só a sequência completa aciona o alerta. Desta forma, os falsos positivos causados por uma descida do telemóvel já ficam corrigidos.

3. Geofencing manual em vez da Google Geofencing API

A API do Google limita a 100 geofences por aplicação e depende de Play Services. Implementar geofencing com cálculo diretamente no `RecordingService` permitiu maior controlo sobre o raio de ativação e entrada/saída da zona.

Distribuição do trabalho dentro da equipa

Tendo em conta o desenvolvimento global do projeto, a contribuição de cada elemento foi distribuída da seguinte forma: Duarte Lourenço contribuiu com 50% do trabalho, e João Silva contribuiu com 50%.

Recursos do projeto (links diretos)

Recurso	Disponível em:
---------	----------------

Code repository:	https://github.com/Duarte-Lourenco/ICM_2025-2026
Ready-to-deploy APK:	https://github.com/Duarte-Lourenco/ICM_2025-2026/releases/tag/v1.0-defesa
App Store page:	<put URL, only if applicable ; store can be Google Play, F-Droid,...>
Demo video:	https://github.com/Duarte-Lourenco/ICM_2025-2026/blob/main/WhatsApp%20Video%202026-05-19%20at%2014.04.07.mp4